Model Quantization

> [!NOTE]
>
> Note: This uses D-Robotics's OpenExplore package and BPU algorithm toolchain. For environment setup, please refer to the official D-Robotics GITHUB documentation. This article will not cover the environment setup process.
>
> **This example uses YOLO's official model yolo11n.pt, quantized to a .bin file**
>
> - The provided virtual machine `Rosmaster-M1-Ubuntu22.04` already has the OE package `horizon_x5_open_explorer_v1.2.8-py310_20240926` and `docker(ubuntu 20.04 cpu version)open_explorer_v1.2.8-py310_20240926` and environment set up.
> - If you need to use GPU docker images, you need to set them up yourself according to the official documentation.

```
D-Robotics OpenExplore(RDK X5, Bayes-e BPU) Version: >= 1.2.8
Ultralytics YOLO Version: 8.3.193
Ubuntu 22.04
Python 3.10
```

1. Introduction

This article is written based on D-Robotics official GITHUB documentation. The prediction program is developed based on rdk_model_zoo. If you need to study and learn, you can search for it yourself.

- Ultralytics YOLO: You Only Look Once
- Introduction to RDK Model Zoo

- Object Detection

2. Preparation

2.1. Environment Preparation and Model Training

Note: This operation needs to be performed on your own x86 machine. It is recommended to use a machine with hardware acceleration for training, such as a GPU supporting CUDA, where torch.cuda.is_available() is True. Ubuntu 22.04, Python 3.10 environment is recommended.

Download the ultralytics/ultralytics repository and refer to the official ultralytics documentation to set up the environment.

For model training, please refer to the official ultralytics documentation. This document is maintained by ultralytics and is of very high quality. There are also many reference materials online. It is not difficult to obtain a model with pre-trained weights like the official one. Please note that no program needs to be modified during training, and the forward method does not need to be modified.

Ultralytics YOLO Official Documentation:

- Quick Start: https://docs.ultralytics.com/quickstart/
- Model Training: https://docs.ultralytics.com/modes/train/

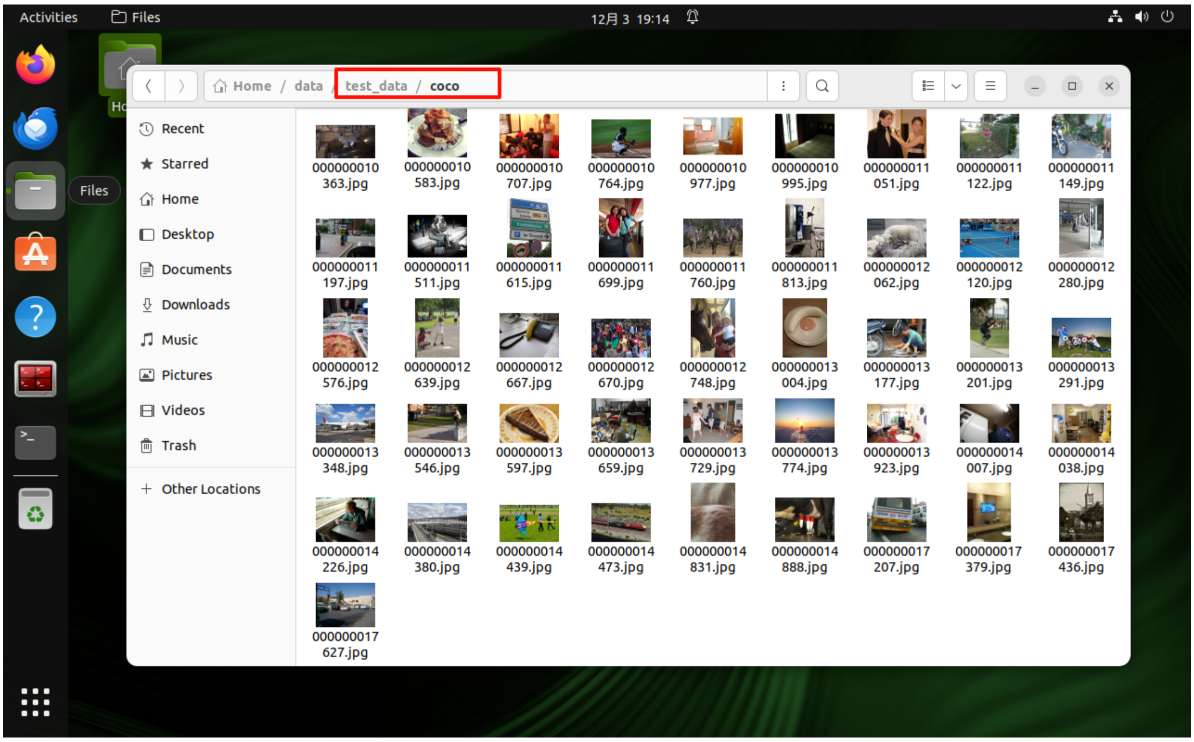
Prepare the `Rosmaster-M1-Ubuntu22.04` virtual machine from the learning materials!

2.2. Prepare Calibration Image Set and PT Model

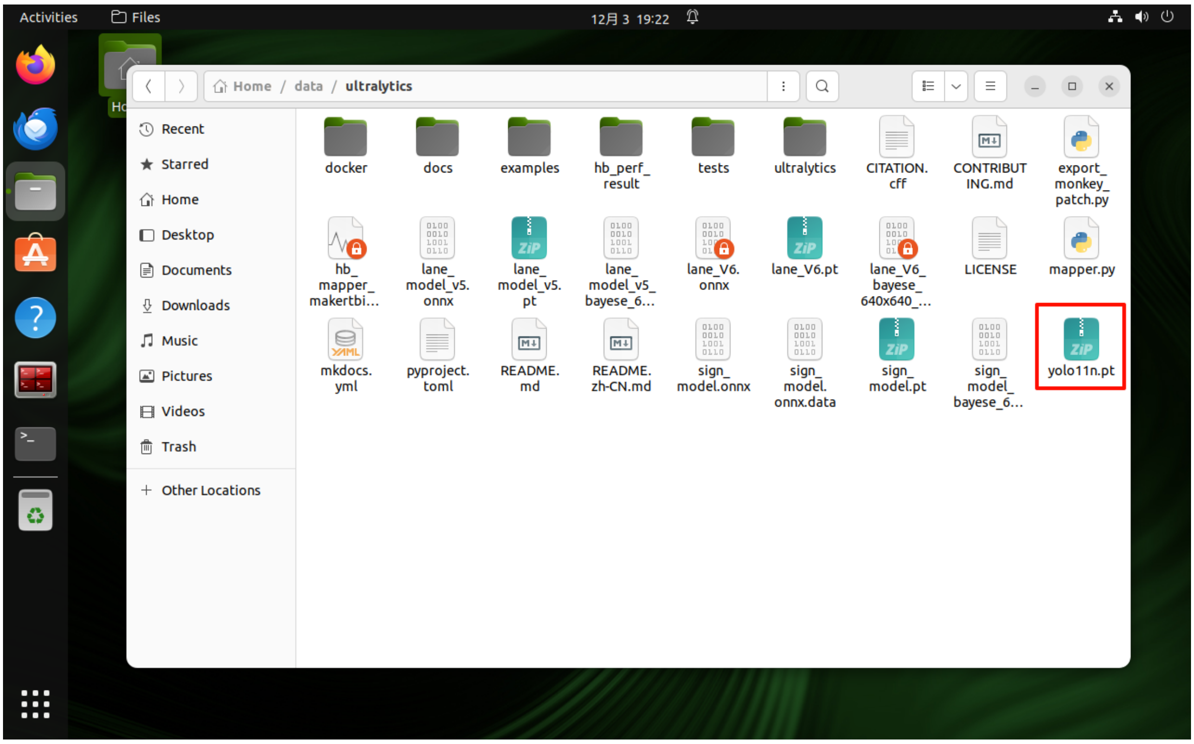
Download `val2017` from the official COCO dataset, and select about 100 images from the validation set - not too many, not too few.

> [!TIP]
>
> The entire preparation work is done within the virtual machine.
>
> Use your own PT model, and use your own validation set images!

- Create a folder named coco yourself and put the images in it



- Put the `yolo11n.pt` model file in this path directory



2.3. Convert ONNX Model

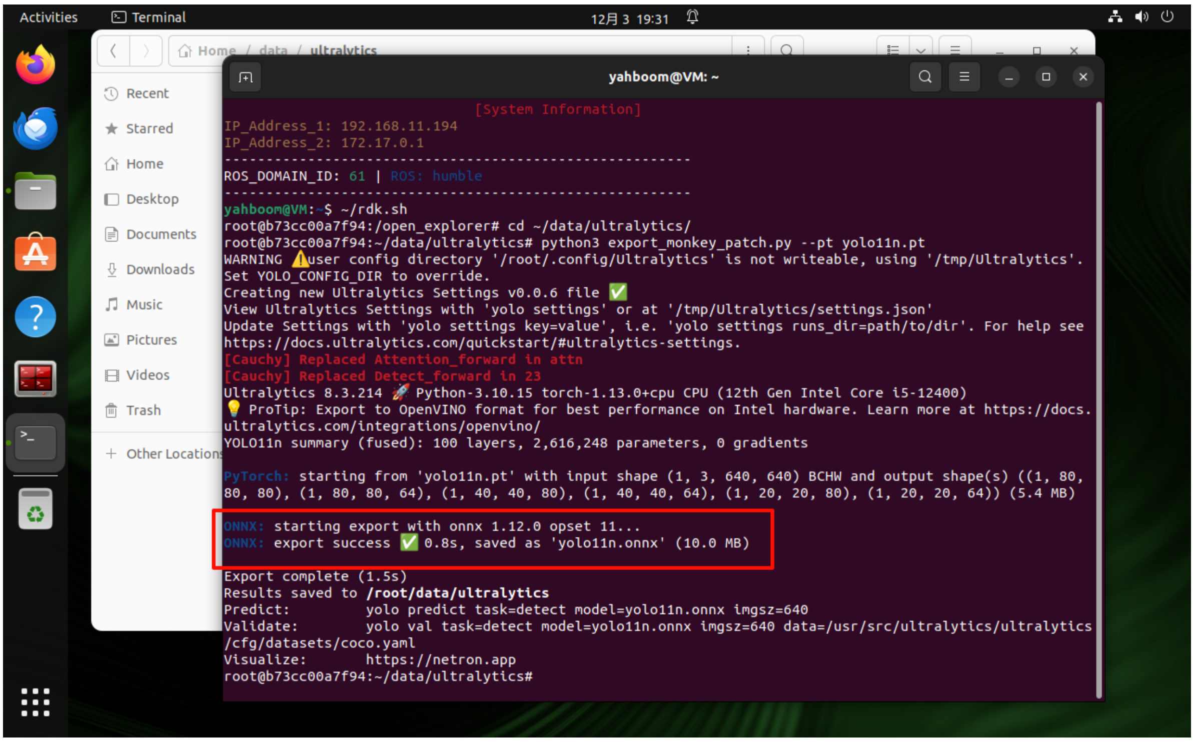
- Open terminal and enter the algorithm toolchain docker container

```
~/rdk.sh
```

- Navigate to the previous path

```
cd ~/data/ultralytics/
```

```
python3 export_monkey_patch.py --pt yolo11n.pt
```

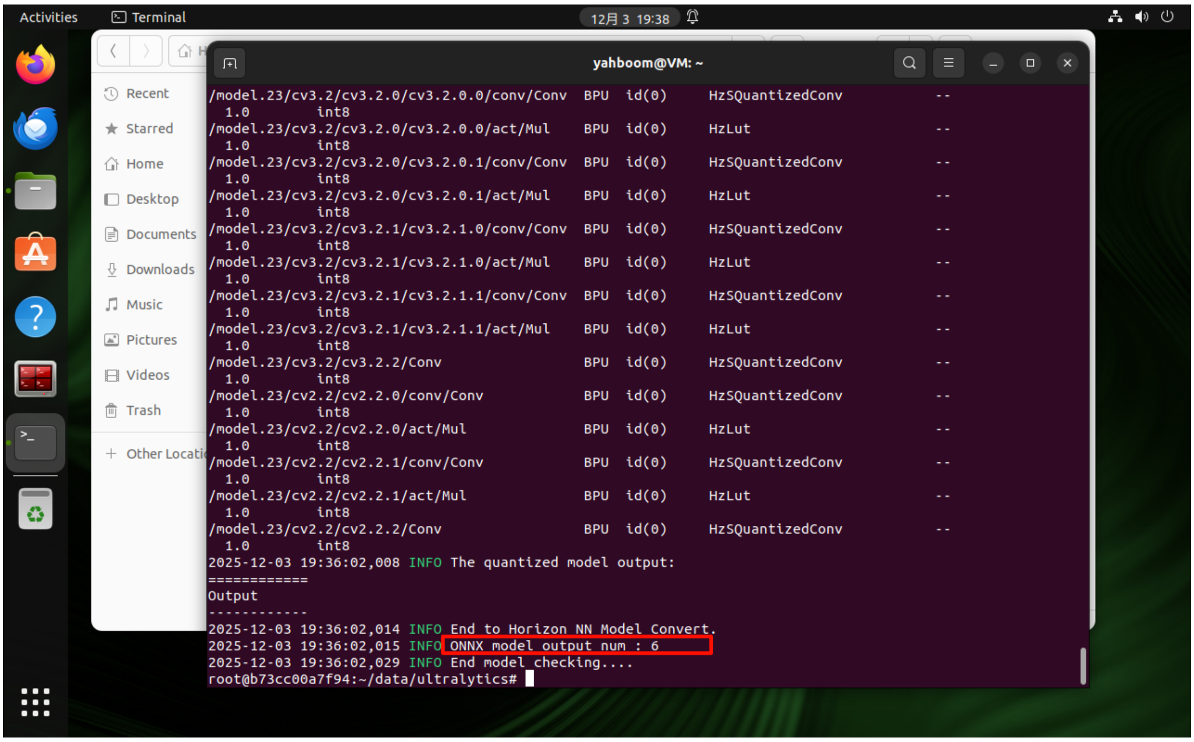


The output shows successful export of the `yolo11n.onnx` model

- Algorithm toolchain model check

```
hb_mapper checker --model-type onnx --march bayes-e --model yolo11n.onnx
```

> [!TIP]
>
> According to the official introduction to this command, we input the command to check our model. The system will have a long output, and we can also see from the output that X5's BPU supports all YOLO operators, meaning all model calculations can be performed on X5's BPU. This check shows six output heads, so we can start the model conversion.

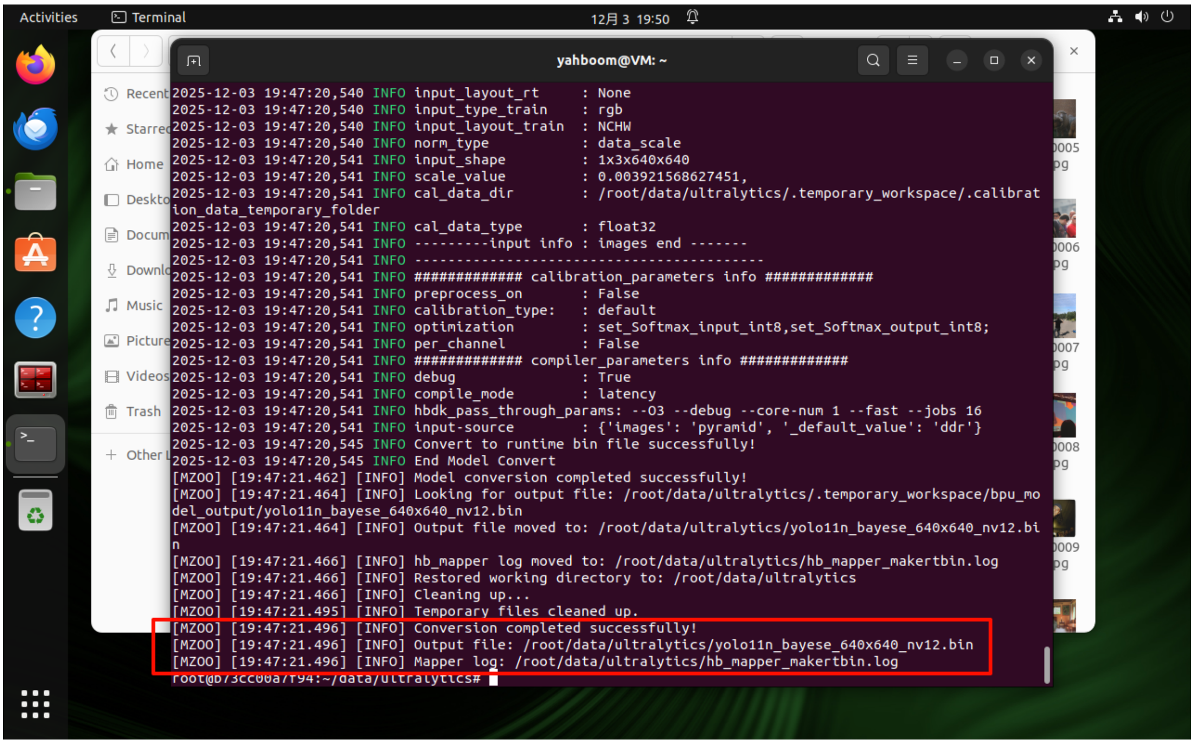


2.4. Quantize Model

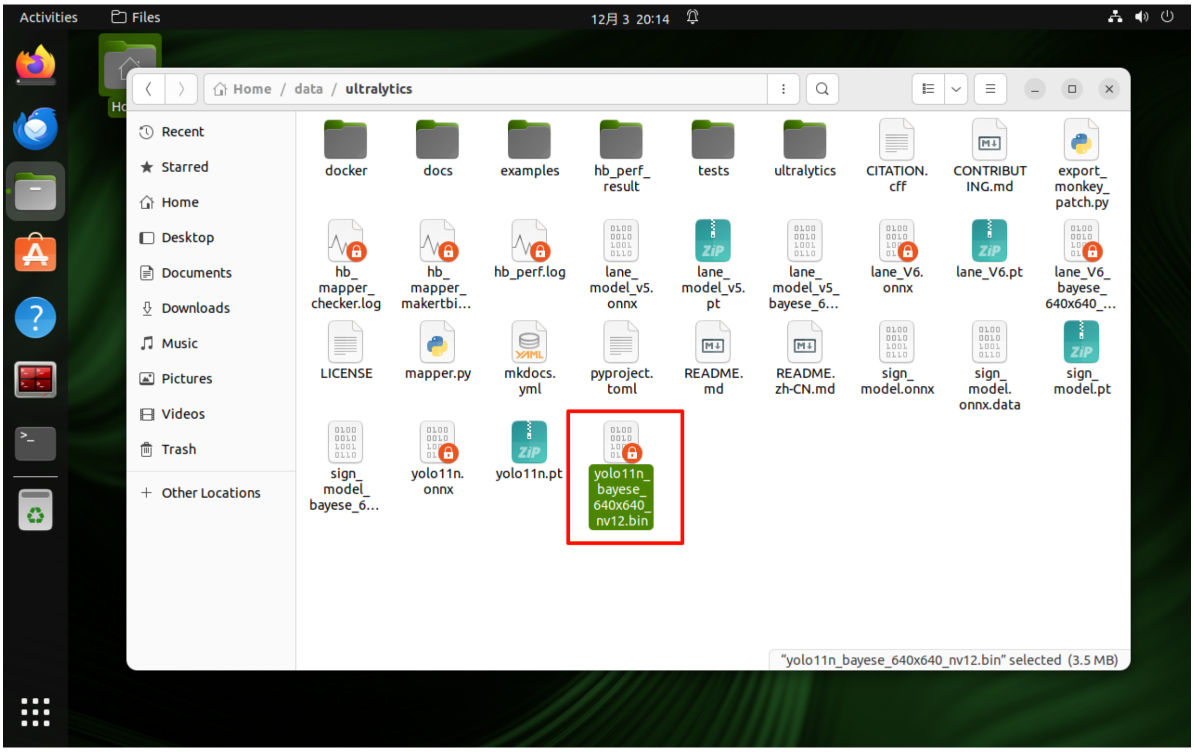
- Start model conversion

```
python3 mapper.py --onnx yolo11n.onnx --cal-images ~/data/test_data/coco
```

> [!TIP]
>
> Fill in the path to your own dataset here
>
> The conversion takes longer for CPU versions depending on model size, please be patient



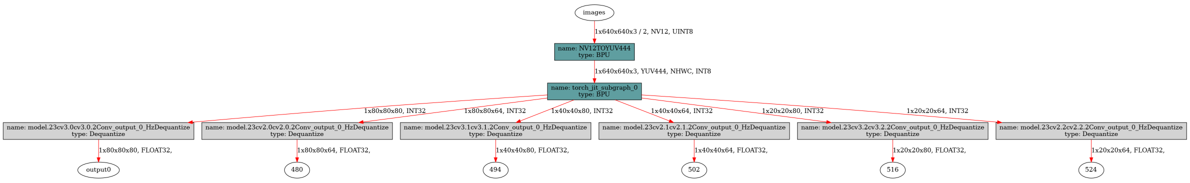
The output shows successful export of the `yolo11n_bayese_640x640_nv12.bin` model



- Algorithm toolchain model check

```
hb_perf yolo11n_bayese_640x640_nv12.bin
```

Then you will get a `hb_perf_result` folder with the verification results. We mainly focus on the model structure diagram inside. The figure below is for reference



3.1. Model Prediction

- (On the robot car host) Enter the ultralytics project

```
cd /home/sunrise/ultralytics/ultralytics
```

> [!TIP]
>
> Put the `yolo11n_bayese_640x640_nv12.bin` obtained from the virtual machine into the project directory on the robot car host
>
> Before running the program, we need to modify the corresponding label names and model path in the program
>
> ```
> # Need to modify
> /home/sunrise/ultralytics/ultralytics/rdk_roscv.py
> ```

```
coco_names = [
    "person", "bicycle", "car", "motorcycle", "airplane", "bus", "train", 
"truck", "boat", "traffic light",
    "fire hydrant", "stop sign", "parking meter", "bench", "bird", "cat", "dog", 
"horse", "sheep", "cow",
    "elephant", "bear", "zebra", "giraffe", "backpack", "umbrella", "handbag", 
"tie", "suitcase", "frisbee",
    "skis", "snowboard", "sports ball", "kite", "baseball bat", "baseball 
glove", "skateboard", "surfboard", "tennis racket", "bottle",
    "wine glass", "cup", "fork", "knife", "spoon", "bowl", "banana", "apple", 
"sandwich", "orange",
    "broccoli", "carrot", "hot dog", "pizza", "donut", "cake", "chair", "couch", 
"potted plant", "bed",
    "dining table", "toilet", "tv", "laptop", "mouse", "remote", "keyboard", 
"cell phone", "microwave", "oven",
    "toaster", "sink", "refrigerator", "book", "clock", "vase", "scissors", 
"teddy bear", "hair drier", "toothbrush"
    ]
```

rdk_roscv.py U ×

ultralytics > ultralytics > rdk_roscv.py

```python
class RDKYOLONode(Node):
    def image_callback(self, msg):
        try:

        except Exception as e:
            self.get_logger().error(f"Error processing image: {e}")

coco_names = [
    "person", "bicycle", "car", "motorcycle", "airplane", "bus", "train", "truck", "boat", "traffic light",
    "fire hydrant", "stop sign", "parking meter", "bench", "bird", "cat", "dog", "horse", "sheep", "cow",
    "elephant", "bear", "zebra", "giraffe", "backpack", "umbrella", "handbag", "tie", "suitcase", "frisbee",
    "skis", "snowboard", "sports ball", "kite", "baseball bat", "baseball glove", "skateboard", "surfboard", "tennis racket", "bottle",
    "wine glass", "cup", "fork", "knife", "spoon", "bowl", "banana", "apple", "sandwich", "orange",
    "broccoli", "carrot", "hot dog", "pizza", "donut", "cake", "chair", "couch", "potted plant", "bed",
    "dining table", "toilet", "tv", "laptop", "mouse", "remote", "keyboard", "cell phone", "microwave", "oven",
    "toaster", "sink", "refrigerator", "book", "clock", "vase", "scissors", "teddy bear", "hair drier", "toothbrush"
    ]
# coco_names = [
#     "greenlight", "honking", "parkA", "parkB", "redlight", "school",
#     "sidewalk", "sidewalk_ground", "speedlimit", "stop", "straight",
#     "turnright", "turnright_ground", "yellowlight"
# ]
# coco_names = [
#     "lane",
# ]
```

rdk_roscv.py U ×

ultralytics > ultralytics > rdk_roscv.py

```python
def draw_detection(img, bbox, score, class_id) -> None:
    label = f"{coco_names[class_id]}: {score:.2f}"
    (label_width, label_height), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
    label_x, label_y = x1, y1 - 10 if y1 - 10 > label_height else y1 + 10
    cv2.rectangle(
        img, (label_x, label_y - label_height), (label_x + label_width, label_y + label_height), color, cv2.FILLED
    )
    cv2.putText(img, label, (label_x, label_y), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1, cv2.LINE_AA)


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--model-path', type=str,
                        default='/home/sunrise/ultralytics/ultralytics/yolo11n_bayese_640x640_nv12.bin',
                        #default='/home/sunrise/ultralytics/ultralytics/lane_V6_bayese_640x640_nv12.bin',
                        help="""Path to BPU Quantized *.bin Model.
                                RDK X3(Module): Bernoulli2.
                                RDK Ultra: Bayes.
                                RDK X5(Module): Bayes-e.
                                RDK S100: Nash-e.
                                RDK S100P: Nash-m.""")
    parser.add_argument('--classes-num', type=int, default=14, help='Classes Num to Detect.')
    parser.add_argument('--nms-thres', type=float, default=0.7, help='IoU threshold.')
    parser.add_argument('--score-thres', type=float, default=0.25, help='confidence threshold.')
    parser.add_argument('--reg', type=int, default=16, help='DFL reg layer.')
    parser.add_argument('--strides', type=lambda s: list(map(int, s.split(','))),
                        default=[8, 16 ,32],
                        help='--strides 8, 16, 32')
    opt = parser.parse_args()
```

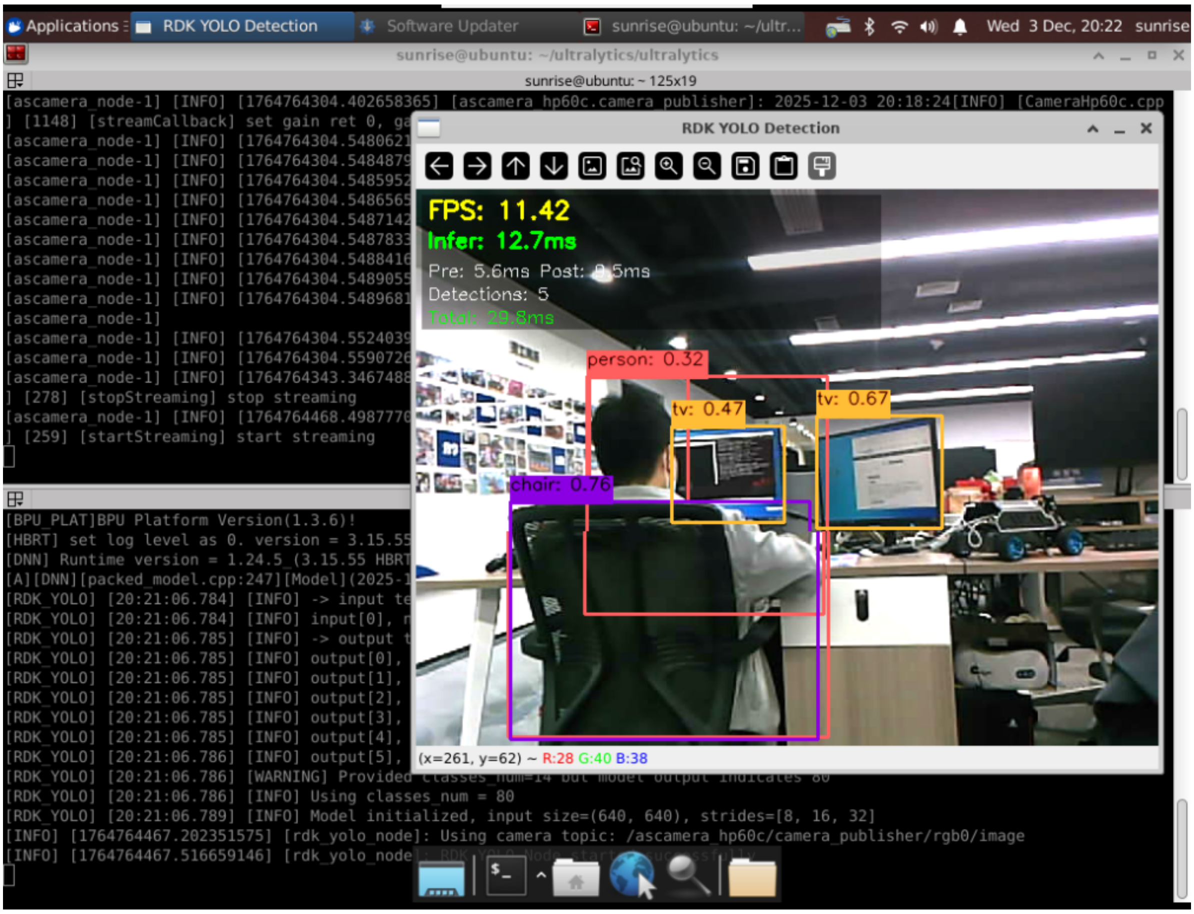
Start the following programs in the terminal according to your camera model,

```
# USB camera
ros2 launch usb_cam camera.launch.py
# Nuwa camera
ros2 launch ascamera hp60c.launch.py

# Prediction program
python3 rdk_roscv.py
```

Effect Preview

> [!TIP]
>
> Taking Nuwa camera as an example,
>
> You can see the inference speed is around 12ms, which is very impressive for the quantized model.


Applications
RDK YOLO Detection
Software Updater
sunrise@ubuntu: ~/ultr...
Wed 3 Dec, 20:22 sunrise
sunrise@ubuntu: ~/ultralytics/ultralytics
sunrise@ubuntu: ~ 125x19
FPS: 11.42
Infer: 12.7ms
Pre: 5.6ms Post: 8.5ms
Detections: 5
Total: 29.8ms
person: 0.32
tv: 0.47
tv: 0.67
chair: 0.76
(x=261, y=62) ~ R:28 G:40 B:38
[BPU_PLAT]BPU Platform Version(1.3.6)!
[RDK_YOLO] [20:21:06.786] [INFO] Using classes_num = 80
[RDK_YOLO] [20:21:06.789] [INFO] Model initialized, input size=(640, 640), strides=[8, 16, 32]
[INFO] [1764764467.202351575] [rdk_yolo_node]: Using camera topic: /ascamera_hp60c/camera_publisher/rgb0/image